gqobrwtn5

March 27, 2026

1 3D-NASE Style Ensemble (SwinUNETR + SegResNet) - Kaggle Optimized

[1]:
```
!pip install -q monai pynrrd tqdm scikit-learn
```

```
                         2.7/2.7 MB
30.1 MB/s eta 0:00:00
```

[2]:
```
import os, torch, numpy as np
from tqdm import tqdm
from sklearn.model_selection import train_test_split

from monai.transforms import *
from monai.data import Dataset, DataLoader
from monai.networks.nets import SwinUNETR, SegResNet
from monai.losses import DiceFocalLoss
from monai.metrics import DiceMetric, HausdorffDistanceMetric
from monai.inferers import sliding_window_inference

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Device:', DEVICE)
```

```
<frozen importlib._bootstrap_external>:1301: FutureWarning: The cuda.cudart
module is deprecated and will be removed in a future release, please switch to
use the cuda.bindings.runtime module instead.
2026-03-27 03:05:48.237186: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1774580748.425658      24 cuda_dnn.cc:8579] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1774580748.481274      24 cuda_blas.cc:1407] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
```

```
already been registered
W0000 00:00:1774580748.952651      24 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1774580748.952692      24 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1774580748.952695      24 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.
W0000 00:00:1774580748.952698      24 computation_placer.cc:177] computation
placer already registered. Please check linkage and avoid linking the same
target more than once.

Device: cuda
```

[3]:
```
image_dir='/kaggle/input/datasets/srujan215/sinus-dataset/NasalSeg/images'
label_dir='/kaggle/input/datasets/srujan215/sinus-dataset/NasalSeg/labels'

files=[]
for f in sorted(os.listdir(image_dir)):
    if f.endswith('_img.nrrd'):
        img=os.path.join(image_dir,f)
        lbl=os.path.join(label_dir,f.replace('_img.nrrd','_seg.nrrd'))
        if os.path.exists(lbl):
            files.append({'image':img,'label':lbl})

train_files,val_files=train_test_split(files,test_size=0.2,random_state=42)
print(len(train_files), len(val_files))
```

```
104 26
```

[4]:
```
# Transforms (ERROR-FREE)
train_transforms = Compose([
    LoadImaged(keys=['image','label']),
    EnsureChannelFirstd(keys=['image','label']),
    ScaleIntensityRanged(keys=['image'], a_min=-1000, a_max=400, b_min=0,␣
 ↪b_max=1, clip=True),
    CropForegroundd(keys=['image','label'], source_key='image'),
    SpatialPadd(keys=['image','label'], spatial_size=(96,96,32)),
    RandCropByPosNegLabeld(keys=['image','label'], label_key='label',␣
 ↪spatial_size=(96,96,32), pos=2, neg=1, num_samples=2),
    ToTensord(keys=['image','label'])
])

val_transforms = Compose([
    LoadImaged(keys=['image','label']),
    EnsureChannelFirstd(keys=['image','label']),
```

```
    ScaleIntensityRanged(keys=['image'], a_min=-1000, a_max=400, b_min=0,␣
 ↪b_max=1, clip=True),
    CropForegroundd(keys=['image','label'], source_key='image'),
    SpatialPadd(keys=['image','label'], spatial_size=(96,96,32)),
    ToTensord(keys=['image','label'])
])
```

[5]:
```
train_loader = DataLoader(Dataset(train_files, train_transforms), batch_size=1,␣
 ↪shuffle=True, num_workers=2)
val_loader = DataLoader(Dataset(val_files, val_transforms), batch_size=1,␣
 ↪num_workers=2)
```

[6]:
```
# Models (Ensemble)
model1 = SwinUNETR(in_channels=1, out_channels=6, feature_size=48).to(DEVICE)
model2 = SegResNet(in_channels=1, out_channels=6, init_filters=32).to(DEVICE)
```

[7]:
```
loss_fn = DiceFocalLoss(to_onehot_y=True, softmax=True)
opt1 = torch.optim.AdamW(model1.parameters(), lr=5e-5, weight_decay=1e-5)
opt2 = torch.optim.AdamW(model2.parameters(), lr=5e-5, weight_decay=1e-5)

dice_metric = DiceMetric(include_background=False, reduction='mean')
hd95_metric = HausdorffDistanceMetric(include_background=False, percentile=95)

scaler = torch.amp.GradScaler('cuda')
```

[8]:
```
# Training Loop (Clean + Stable)
EPOCHS = 50
best_dice = 0

for epoch in range(EPOCHS):
    model1.train(); model2.train()
    loss_sum = 0

    for b in train_loader:
        img=b['image'].to(DEVICE)
        lbl=b['label'].to(DEVICE)

        opt1.zero_grad(); opt2.zero_grad()

        with torch.amp.autocast('cuda'):
            p1=model1(img)
            p2=model2(img)
            loss = loss_fn(p1,lbl) + loss_fn(p2,lbl)

        scaler.scale(loss).backward()
        scaler.step(opt1); scaler.step(opt2)
        scaler.update()
```

```
        loss_sum += loss.item()

    model1.eval(); model2.eval()
    dice_metric.reset(); hd95_metric.reset()

    with torch.no_grad():
        for b in val_loader:
            img=b['image'].to(DEVICE)
            lbl=b['label'].to(DEVICE)

            p1 = sliding_window_inference(img,(96,96,32),2,model1)
            p2 = sliding_window_inference(img,(96,96,32),2,model2)

            pred = (p1+p2)/2
            pred = torch.argmax(pred,1,keepdim=True)

            dice_metric(y_pred=pred, y=lbl)
            hd95_metric(y_pred=pred.cpu(), y=lbl.cpu())

    dice = dice_metric.aggregate().item()
    hd = hd95_metric.aggregate().item()

    dice_metric.reset(); hd95_metric.reset()

    print(f"Epoch {epoch+1} | Loss {loss_sum:.3f} | Dice {dice:.4f} | HD95 {hd:.
↪2f}")

    if dice > best_dice:
        best_dice = dice
        torch.save(model1.state_dict(), '/kaggle/working/best_swin.pth')
        torch.save(model2.state_dict(), '/kaggle/working/best_seg.pth')

print('Best Dice:', best_dice)
```

```
/usr/local/lib/python3.12/dist-packages/monai/inferers/utils.py:226:
UserWarning: Using a non-tuple sequence for multidimensional indexing is
deprecated and will be changed in pytorch 2.9; use x[tuple(seq)] instead of
x[seq]. In pytorch 2.9 this will be interpreted as tensor index,
x[torch.tensor(seq)], which will result either in an error or a different result
(Triggered internally at
/pytorch/torch/csrc/autograd/python_variable_indexing.cpp:347.)
  win_data = torch.cat([inputs[win_slice] for win_slice in
unravel_slice]).to(sw_device)
/usr/local/lib/python3.12/dist-packages/monai/inferers/utils.py:370:
UserWarning: Using a non-tuple sequence for multidimensional indexing is
deprecated and will be changed in pytorch 2.9; use x[tuple(seq)] instead of
```

```
x[seq]. In pytorch 2.9 this will be interpreted as tensor index,
x[torch.tensor(seq)], which will result either in an error or a different result
(Triggered internally at
/pytorch/torch/csrc/autograd/python_variable_indexing.cpp:347.)
  out[idx_zm] += p
/usr/local/lib/python3.12/dist-packages/monai/utils/deprecate_utils.py:221:
FutureWarning: monai.metrics.utils get_mask_edges:always_return_as_numpy:
Argument `always_return_as_numpy` has been deprecated since version 1.5.0. It
will be removed in version 1.7.0. The option is removed and the return type will
always be equal to the input type.
  warn_deprecated(argname, msg, warning_category)
/usr/local/lib/python3.12/dist-packages/monai/inferers/utils.py:226:
UserWarning: Using a non-tuple sequence for multidimensional indexing is
deprecated and will be changed in pytorch 2.9; use x[tuple(seq)] instead of
x[seq]. In pytorch 2.9 this will be interpreted as tensor index,
x[torch.tensor(seq)], which will result either in an error or a different result
(Triggered internally at
/pytorch/torch/csrc/autograd/python_variable_indexing.cpp:347.)
  win_data = torch.cat([inputs[win_slice] for win_slice in
unravel_slice]).to(sw_device)
/usr/local/lib/python3.12/dist-packages/monai/inferers/utils.py:370:
UserWarning: Using a non-tuple sequence for multidimensional indexing is
deprecated and will be changed in pytorch 2.9; use x[tuple(seq)] instead of
x[seq]. In pytorch 2.9 this will be interpreted as tensor index,
x[torch.tensor(seq)], which will result either in an error or a different result
(Triggered internally at
/pytorch/torch/csrc/autograd/python_variable_indexing.cpp:347.)
  out[idx_zm] += p
/usr/local/lib/python3.12/dist-packages/monai/utils/deprecate_utils.py:221:
FutureWarning: monai.metrics.utils get_mask_edges:always_return_as_numpy:
Argument `always_return_as_numpy` has been deprecated since version 1.5.0. It
will be removed in version 1.7.0. The option is removed and the return type will
always be equal to the input type.
  warn_deprecated(argname, msg, warning_category)

Epoch 1 | Loss 201.840 | Dice 0.3444 | HD95 86.73
Epoch 2 | Loss 176.859 | Dice 0.6259 | HD95 96.35
Epoch 3 | Loss 157.430 | Dice 0.7840 | HD95 91.32
Epoch 4 | Loss 145.901 | Dice 0.7593 | HD95 87.34
Epoch 5 | Loss 135.776 | Dice 0.8241 | HD95 85.57
Epoch 6 | Loss 129.997 | Dice 0.8145 | HD95 85.68
Epoch 7 | Loss 122.012 | Dice 0.9078 | HD95 33.72
Epoch 8 | Loss 113.313 | Dice 0.9202 | HD95 36.11
Epoch 9 | Loss 108.055 | Dice 0.9240 | HD95 28.84
Epoch 10 | Loss 99.777 | Dice 0.9300 | HD95 23.15
Epoch 11 | Loss 95.660 | Dice 0.9351 | HD95 21.99
Epoch 12 | Loss 92.575 | Dice 0.9348 | HD95 9.79
Epoch 13 | Loss 88.182 | Dice 0.9463 | HD95 4.62
```

```
Epoch 14 | Loss 81.550 | Dice 0.9434 | HD95 4.03
Epoch 15 | Loss 79.124 | Dice 0.9468 | HD95 9.43
Epoch 16 | Loss 76.775 | Dice 0.9542 | HD95 10.31
Epoch 17 | Loss 74.270 | Dice 0.9561 | HD95 10.58
Epoch 18 | Loss 72.169 | Dice 0.9560 | HD95 1.43
Epoch 19 | Loss 68.425 | Dice 0.9568 | HD95 4.37
Epoch 20 | Loss 68.775 | Dice 0.9592 | HD95 1.44
Epoch 21 | Loss 63.924 | Dice 0.9607 | HD95 1.18
Epoch 22 | Loss 66.233 | Dice 0.9525 | HD95 4.10
Epoch 23 | Loss 60.858 | Dice 0.9581 | HD95 2.21
Epoch 24 | Loss 61.025 | Dice 0.9590 | HD95 2.60
Epoch 25 | Loss 64.480 | Dice 0.9527 | HD95 3.80
Epoch 26 | Loss 60.164 | Dice 0.9512 | HD95 9.51
Epoch 27 | Loss 59.318 | Dice 0.9572 | HD95 2.27
Epoch 28 | Loss 57.346 | Dice 0.9599 | HD95 1.35
Epoch 29 | Loss 57.316 | Dice 0.9621 | HD95 1.35
Epoch 30 | Loss 57.767 | Dice 0.9573 | HD95 2.29
Epoch 31 | Loss 56.803 | Dice 0.9580 | HD95 1.51
Epoch 32 | Loss 54.851 | Dice 0.9618 | HD95 2.01
Epoch 33 | Loss 51.970 | Dice 0.9609 | HD95 2.23
Epoch 34 | Loss 54.550 | Dice 0.9602 | HD95 2.18
Epoch 35 | Loss 53.325 | Dice 0.9559 | HD95 2.99
Epoch 36 | Loss 52.268 | Dice 0.9572 | HD95 1.37
Epoch 37 | Loss 53.675 | Dice 0.9611 | HD95 1.35
Epoch 38 | Loss 51.824 | Dice 0.9638 | HD95 1.33
Epoch 39 | Loss 52.693 | Dice 0.9606 | HD95 3.78
Epoch 40 | Loss 52.344 | Dice 0.9594 | HD95 4.30
Epoch 41 | Loss 52.744 | Dice 0.9612 | HD95 1.33
Epoch 42 | Loss 48.585 | Dice 0.9628 | HD95 1.33
Epoch 43 | Loss 48.145 | Dice 0.9550 | HD95 2.35
Epoch 44 | Loss 51.122 | Dice 0.9616 | HD95 4.04
Epoch 45 | Loss 49.603 | Dice 0.9635 | HD95 1.27
Epoch 46 | Loss 47.490 | Dice 0.9627 | HD95 1.31
Epoch 47 | Loss 49.164 | Dice 0.9642 | HD95 1.19
Epoch 48 | Loss 50.626 | Dice 0.9645 | HD95 1.12
Epoch 49 | Loss 49.667 | Dice 0.9629 | HD95 1.34
Epoch 50 | Loss 50.328 | Dice 0.9645 | HD95 1.29
Best Dice: 0.96451336145401
```